System Architecture Document

Arati Business ERP

Document Version: 2.0
Status: Architecture Baseline — Draft for Implementation
Project: Arati Business ERP
Primary Scope: Distribution + Wholesale + Retail + Inventory ERP
Accounting Integration: TallyPrime
Priority: ERP Software First; Public Website Later

Related Documents

- `BRD.md`
- `SRS.md`
- `PROTECTION.md`
- `TECHSTACK_v1.1.md`
- `BUSINESS-RULES.md`
- `DATABASE.md`
- `API.md`
- `TALLY-INTEGRATION.md`
- `DESIGN.md`
- `PHASES.md`

---

1. Purpose

This document defines the system architecture for the Arati Business ERP.

It explains how the system is structured, how modules interact, where business rules live, how transactions remain consistent, how inventory is protected, how security is enforced, and how external systems such as TallyPrime are isolated from the ERP core.

This document intentionally does **not** replace:

- `DATABASE.md` for detailed database schema
- `API.md` for endpoint contracts
- `TALLY-INTEGRATION.md` for exact Tally mappings
- `PROTECTION.md` for detailed security controls
- `TECHSTACK_v1.1.md` for technology choices

- `BUSINESS-RULES.md` for detailed business policy
- `DESIGN.md` for UI/UX
- `PHASES.md` for delivery planning

Architecture decisions in this document are binding unless superseded by an approved Architecture Decision Record (ADR).

---

2. Architectural Objectives

The architecture shall prioritize:

1. Transaction correctness
2. Inventory integrity
3. Financial-data integrity
4. Security and authorization
5. Auditability
6. Tally integration resilience
7. Maintainability
8. Operational simplicity
9. Performance appropriate for retail/POS workloads
10. Controlled extensibility
11. Safe failure and recovery
12. Future multi-branch capability

The architecture must be strong enough for production use without introducing infrastructure that is not justified by actual requirements.

---

3. Core Architectural Principles

3.1 Generic by Design

The software must not contain business logic hard-coded to specific:

- Brands
- Suppliers
- Customers
- Dealers
- Products
- Models
- Price structures

Business-specific behavior belongs in master data, configuration, permissions, pricing, workflows, and approved business rules.

3.2 PostgreSQL Is the Transactional Source of Truth

PostgreSQL is authoritative for ERP operational and transactional state.

This includes:

- Products
- Parties
- Purchases
- Sales
- Inventory
- Payments
- Returns
- Deliveries
- Installations
- Warranty records
- Audit records
- Integration state

The following must never become the authoritative source of inventory or financial state:

- Browser state
- Frontend caches
- Redis
- Queue payloads
- Tally
- Search indexes
- Temporary files

3.3 ERP Is the Operational Source of Truth

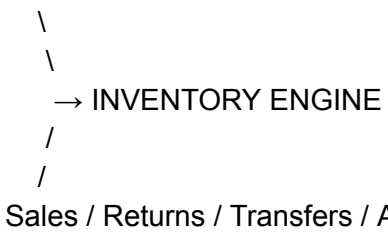
The ERP owns operational business state.

TallyPrime is an external accounting system. It may receive accounting-relevant information from the ERP, but ERP operations must not depend on Tally being available.

3.4 Inventory Is a Core Domain

Inventory is not merely a side effect of sales and purchases.

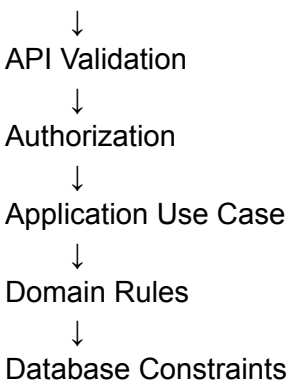
All stock-affecting operations must pass through controlled inventory logic.



No module may independently modify stock balances.

3.5 Business Rules Are Server-Enforced

The frontend may provide validation and usability assistance, but it is never authoritative.



Critical rules must remain enforced when requests are made through:

- Web UI
- POS UI
- Internal APIs
- Imports
- Background workers
- Integrations

3.6 External Systems Must Not Block Core Transactions

A valid ERP transaction must be able to complete even if Tally, email, SMS, WhatsApp, or another external service is unavailable.

ERP Transaction
↓
Database Commit
↓
Integration/Event Record
↓
Background Processing
↓
External System

3.7 Posted Transactions Are Historical Facts

Important posted business transactions must not be silently deleted.

Corrections should use:

- Cancellation
- Reversal
- Return
- Adjustment
- Corrective transaction

according to the applicable business rule.

3.8 No Premature Microservices

The initial architecture is a modular monolith.

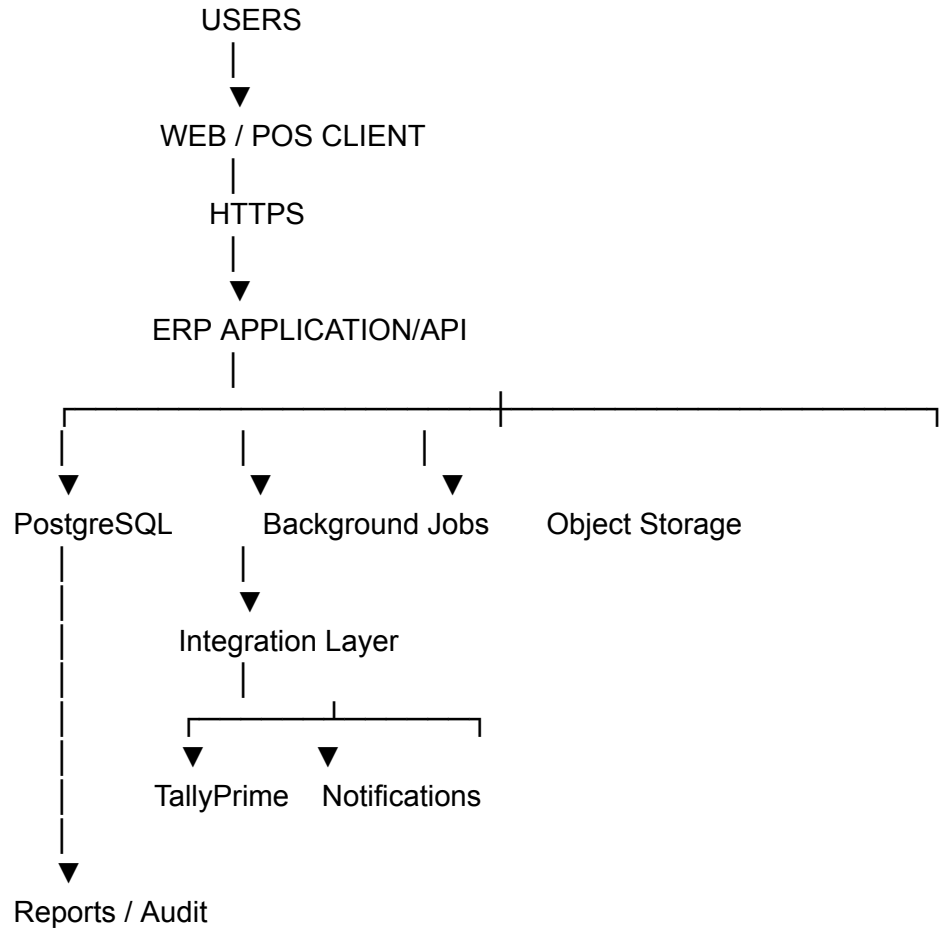
Service extraction is permitted only when there is a demonstrated need such as:

- Independent scaling
- Independent deployment
- Operational isolation
- Team ownership requirements
- Reliability boundaries

Microservices are not an architectural goal by themselves.

---

4. System Context



Potential future integrations:

- WhatsApp
- SMS
- Email
- Payment gateways
- E-commerce
- Logistics providers
- External service/warranty systems

All integrations must be isolated behind explicit boundaries.

---

5. System Boundaries

5.1 Inside ERP

The ERP owns:

- Authentication context
- Users and permissions
- Organization structure
- Product master
- Party master
- Pricing
- Procurement
- Inventory
- Sales/POS
- Payments and allocations
- Returns
- Fulfilment
- Delivery
- Installation
- Warranty tracking
- Operational reporting
- Audit
- Integration orchestration

5.2 Outside ERP

External systems own their own concerns.

Examples:

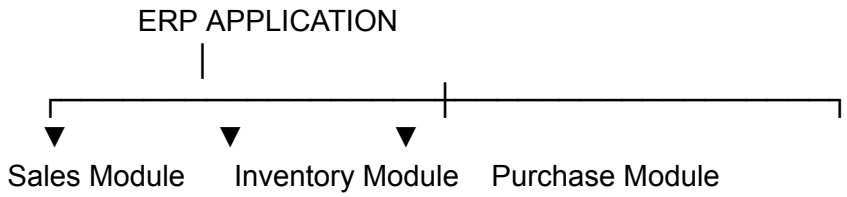
- TallyPrime → accounting system
- Object storage → binary file storage
- Email/SMS/WhatsApp → message delivery
- Payment gateway → external payment processing

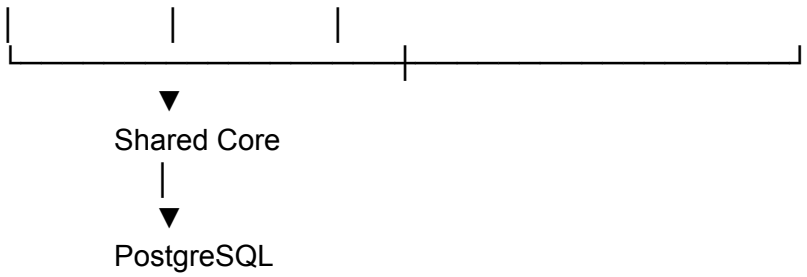
The ERP stores the minimum integration state required for reliable operation and reconciliation.

---

6. Architectural Style

The system shall use a **modular monolith**.





The modules share one application deployment and one primary transactional database while maintaining strict logical ownership.

This provides strong transactions without the operational complexity of distributed services.

---

7. Internal Module Boundaries

Recommended module boundaries:

AUTH
ORGANIZATION
PARTIES
PRODUCTS
PRICING
PROCUREMENT
INVENTORY
SALES
PAYMENTS
RETURNS
FULFILMENT
DELIVERY
INSTALLATION
WARRANTY
REPORTING
AUDIT
NOTIFICATIONS
INTEGRATIONS
TALLY
SETTINGS

A module owns its domain behavior and persistence access.

Other modules must interact through:

- Application services
- Domain services
- Explicit interfaces

- Domain/application events where appropriate

A module must not casually update another module's tables.

---

8. Domain Ownership

Domain	Primary Owner	Examples
Authentication	Auth	Login, sessions, password policy
Authorization	Auth	Roles, permissions
Organization	Organization	Branches, warehouses
Party	Parties	Customer, dealer, supplier
Product	Products	Brand, category, product, variant
Pricing	Pricing	Price lists, party pricing
Procurement	Procurement	PO, GRN, purchase invoice
Inventory	Inventory	Stock movement, balance, serials
Sales	Sales	Quotation, order, invoice, POS
Payments	Payments	Receipts, allocations
Returns	Returns	Sales/purchase returns
Fulfilment	Fulfilment	Fulfilment state and allocation
Delivery	Delivery	Delivery lifecycle
Installation	Installation	Installation lifecycle
Warranty	Warranty	Warranty registration/claims
Reporting	Reporting	Operational reports
Audit	Audit	Audit trail
Tally	Tally/Integrations	Sync and reconciliation

---

9. Module Dependency Rules

Dependency direction should generally be:

Presentation
↓
Application
↓
Domain
↓
Persistence / Infrastructure

Cross-module dependency must be intentional.

Examples:

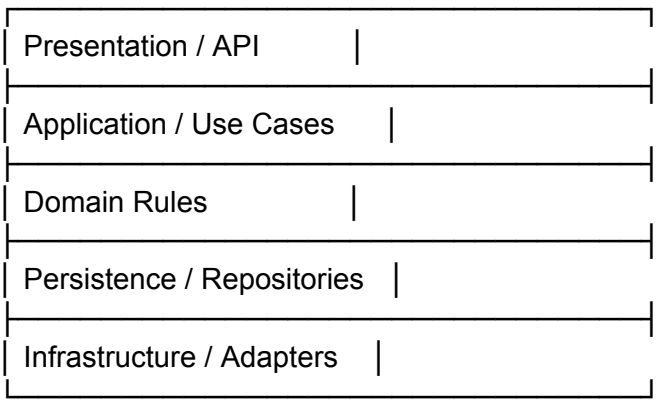
Sales → Pricing
Sales → Parties
Sales → Inventory
Sales → Payments
Sales → Fulfilment
Procurement → Parties
Procurement → Products
Procurement → Inventory
Procurement → Payments
Returns → Sales
Returns → Inventory
Returns → Payments

Circular dependencies must be avoided where practical.

When two modules need to react to each other, use application events or a shared orchestration layer rather than direct table manipulation.

---

10. Layered Application Architecture



10.1 Presentation Layer

Responsible for:

- HTTP
- Authentication context extraction
- Request parsing
- Input validation
- Response formatting
- API documentation

It must not contain authoritative business decisions.

10.2 Application Layer

Responsible for:

- Use cases
- Transaction orchestration
- Permission checks
- Workflow coordination
- Calling domain services
- Coordinating multiple modules

10.3 Domain Layer

Responsible for:

- Business rules
- State transitions
- Pricing rules
- Inventory rules
- Financial allocation rules
- Invariants

10.4 Persistence Layer

Responsible for:

- Repositories
- Queries
- Persistence mapping
- Transaction participation
- Database-specific operations

10.5 Infrastructure Layer

Responsible for:

- Tally adapters
- Queue adapters
- Object storage
- Email/SMS/WhatsApp providers
- External APIs
- Observability integrations

---

11. Request Lifecycle

A normal authenticated request should follow:

Client
↓
HTTPS
↓
Authentication
↓
Rate / Request Controls
↓
Input Validation
↓
Authorization
↓
Application Use Case
↓
Domain Rules
↓
Database Transaction
↓
Audit / Integration Event
↓
Commit
↓
Response

For asynchronous work:

Committed Transaction
↓
Job / Event
↓
Worker
↓

External Operation
↓
Result
↓
Retry / Success / Manual Review

---

12. Transaction Architecture

Critical business operations must be atomic.

Example sale:

BEGIN TRANSACTION

Create / update sale
Create sale items
Validate pricing
Validate customer/party
Validate stock
Create inventory movements
Update inventory projection
Update serial state where applicable
Record payment
Create payment allocations
Create audit event
Create integration/outbox record

COMMIT

If a critical step fails, the transaction must roll back.

External communication should normally happen after the ERP transaction is safely committed.

---

13. Transaction Boundary Rules

13.1 Must Be Atomic

Where applicable:

- Sale + stock deduction
- Purchase receipt + stock addition

- Stock transfer out + transfer in
- Return + inventory effect
- Payment + allocation
- Serial assignment + stock movement
- Stock adjustment + movement
- Cancellation + reversal effects

13.2 Should Not Be Part of the Core Transaction

Unless there is a deliberate architectural reason:

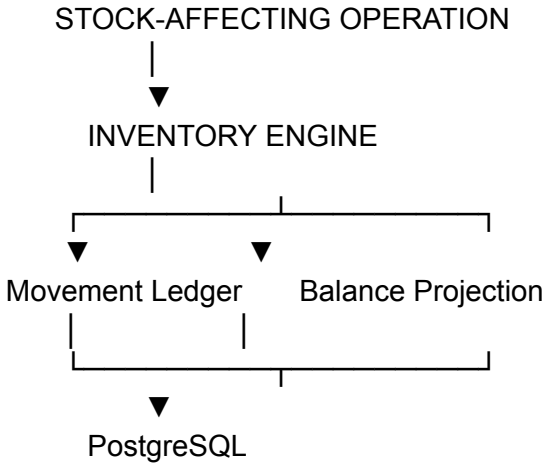
- Tally HTTP communication
- Email sending
- SMS sending
- WhatsApp sending
- Large report generation
- External API calls
- File processing

These should normally use post-commit jobs/events.

---

14. Inventory Architecture

Inventory is controlled through an append-oriented stock movement ledger plus a balance projection.



The ledger provides traceability.

The balance projection provides efficient current-stock reads.

The projection is rebuildable from authoritative movement history where the implemented design permits.

---

15. Inventory Movement Sources

Stock movement may originate from:

- Opening stock
- Goods receipt
- Sales issue
- Sales return
- Purchase return
- Stock transfer
- Stock adjustment
- Other explicitly approved stock events

Every movement should be traceable to a business reason/source.

---

16. Inventory Invariants

The inventory engine must protect at least:

1. No unauthorized stock movement.
2. No duplicate processing of the same stock-affecting operation.
3. No accidental double deduction.
4. Serialized units cannot be sold twice.
5. Serial/batch/location relationships remain consistent.
6. Stock transfer cannot silently lose or duplicate stock.
7. Returned goods do not automatically become sellable without the configured disposition.
8. Negative stock behavior follows the approved business rule.
9. Stock-changing operations are auditable.
10. Inventory balances cannot be trusted if the corresponding transaction is not committed.

Rules whose business policy is not yet confirmed must remain explicitly marked:

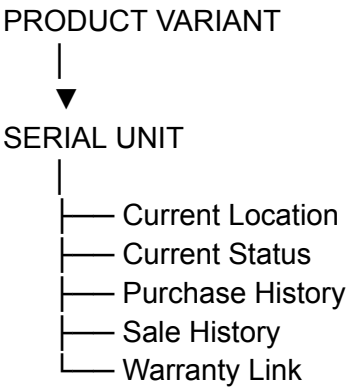
```
TBD — Business Confirmation Required
```

---

17. Serialized Inventory

Serialized inventory requires stronger controls than quantity-only stock.

Conceptually:



A serial unit must have a traceable lifecycle.

A sale of a serialized unit must validate that the unit is eligible for sale and is available at the required location.

Concurrent operations must prevent the same serial from being consumed twice.

---

18. Batch Inventory

Batch-tracked products must preserve batch identity through stock movements where required.

A batch may carry information such as:

- Batch number
- Product variant
- Manufacturing/expiry information where applicable
- Quantity
- Location
- Source transaction

If one document line can contain multiple batches, the data model must use an allocation structure rather than assuming one batch per document line.

---

19. Stock Transfers

A transfer represents movement between inventory locations.

SOURCE LOCATION
↓
TRANSFER OUT
↓
TRANSFER TRANSACTION
↓
TRANSFER IN
↓
DESTINATION LOCATION

The implementation must define whether transfer states allow:

- Draft
- Approved
- In transit
- Received
- Cancelled

A transfer must not create or destroy stock.

---

20. Stock Adjustments

Adjustments are controlled exceptions, not normal inventory operations.

They should require:

- Authorized permission
- Reason
- Quantity
- Location
- Product/variant
- Appropriate serial/batch information
- Audit record

If approval thresholds are required, those thresholds remain:

`TBD — Business Confirmation Required`

---

21. Concurrency Control

Inventory correctness must survive concurrent users.

Example:

Stock = 1

User A → attempts sale
User B → attempts sale

Only one operation may successfully consume the available unit when negative stock is not permitted.

The implementation may use:

- Database row locking
- Atomic conditional updates
- Serializable/appropriate transaction isolation
- Unique constraints
- Serial-row locking

The exact mechanism belongs in implementation and database design, but the invariant is architectural.

---

22. Financial Transaction Architecture

The ERP may maintain operational financial records needed for:

- Invoice totals
- Payments
- Receivables/payables
- Payment allocations
- Credit status
- Tally integration

It does not attempt to recreate TallyPrime's complete accounting engine in the initial release.

Money values must use exact decimal/numeric representations rather than floating-point arithmetic.

---

23. Payment Architecture

Payments are independent records.

PAYMENT

```
│
├── Party
├── Amount
├── Method
├── Date
├── Reference
│
└── ALLOCATIONS
      ├── Invoice A
      ├── Invoice B
      └── Invoice C
```

The architecture supports:

- One payment → multiple invoices
- One invoice → multiple payments
- Partial payment
- Unallocated/advance payment where business rules permit

Over-allocation must be prevented.

---

24. Sales Architecture

A single sales engine supports:

```
SALES
├── Retail
├── Wholesale
└── Dealer
```

Differences are represented through:

- Sale type
- Party
- Price list
- Payment terms
- Credit rules
- Discounts
- Fulfilment behavior
- Configuration

POS is a specialized frontend/workflow over the same sales domain.

There must not be separate retail and wholesale inventory engines.

---

25. Purchase Architecture

The architecture supports:

Purchase Order
↓
Goods Receipt
↓
Purchase Invoice
↓
Supplier Payable
↓
Payment

The workflow may be simplified where the business does not require every document.

Partial receipts and other real-world relationships must be supported if confirmed by business requirements.

---

26. Pricing Architecture

Pricing is a dedicated domain.

Conceptually:

Product
+
Party
+
Sale Type
+
Price List
+
Quantity
+
Discount / Promotion
↓
Applicable Price

Pricing decisions must be deterministic and traceable.

The product master must not be mutated simply because a different customer receives a different price.

Pricing precedence rules must be documented in `BUSINESS-RULES.md`.

---

27. Returns Architecture

Returns are independent transactions linked to their source transaction where possible.

ORIGINAL TRANSACTION
↓
RETURN
↓
INSPECTION
↓
DISPOSITION
↓
Inventory / Financial Effect

Possible disposition values may include:

- RESELLABLE
- REPAIR_REQUIRED
- DAMAGED
- SCRAP
- SUPPLIER_RETURN

The exact allowed states and financial effects must be confirmed in business rules.

---

28. Fulfilment, Delivery and Installation

Sales and fulfilment are separate concerns.

SALE
↓
FULFILMENT
↓
DELIVERY
↓
INSTALLATION (optional)

This supports:

- Partial fulfilment
- Delivery status
- Delivery assignment
- Installation tracking

The architecture should support multiple items/units per delivery or installation where required.

Billing completion must not be incorrectly equated with physical delivery completion.

---

29. Warranty Architecture

Warranty is linked to the applicable product and sale context.

For serialized products:

SALE
↓
SERIAL UNIT
↓
WARRANTY
↓
SERVICE / CLAIM

Warranty start-date policy is:

`TBD – Business Confirmation Required`

Advanced service management can be introduced later without redesigning the core product/sale/serial relationships.

---

30. Organization and Location Architecture

The organization hierarchy is:

ORGANIZATION
│
├── BRANCH
│ │
│ └── WAREHOUSE
│ │
│ └── STOCK LOCATION
│
└── Users / Configuration

The initial deployment may contain one organization, one branch, and one warehouse.

The architecture must not require redesign when additional branches or warehouses are introduced.

---

31. Multi-Branch Data Isolation

Branch/organization context must be enforced server-side.

A user authorized for one branch must not automatically gain access to another branch's operational data.

Authorization may combine:

- User
- Role
- Permission
- Organization
- Branch
- Warehouse/location scope

This is an object-level authorization concern, not merely a menu-visibility concern.

---

32. Master Data Lifecycle

Master data generally uses controlled states such as:

ACTIVE
INACTIVE

Referenced historical masters should normally be deactivated rather than deleted.

Examples:

- Product
- Party
- Price list
- Warehouse
- Tax rule

Historical transactions must remain readable.

---

33. Document Lifecycle

Business documents should use explicit state transitions.

Example:

DRAFT
↓
CONFIRMED
↓
POSTED
↓
COMPLETED

Other terminal or exceptional states may include:

CANCELLED
REJECTED
FAILED
REVERSED

Not every document requires every state.

State transitions must be controlled by domain rules and permissions.

---

34. Authentication Architecture

Authentication is centralized.

User
↓
Credential / Authentication
↓
Session
↓
Authenticated Identity
↓
Authorization Context
↓
API

The preferred implementation should use secure server-managed session mechanisms where appropriate.

Authentication must not be implemented as a collection of unrelated frontend checks.

---

35. Authorization Architecture

Use RBAC with granular permissions.

USER
↓
ROLE
↓
PERMISSIONS

Examples:

SALE_CREATE
SALE_CANCEL
PURCHASE_CREATE
INVENTORY_VIEW
INVENTORY_ADJUST
PAYMENT_CREATE
PAYMENT_REVERSE
REPORT_VIEW
TALLY_RETRY
TALLY_RECONCILE
USER_MANAGE

Permission names are illustrative; the canonical permission catalog belongs in security/application design.

Authorization must be checked server-side.

---

36. Object-Level Authorization

A valid permission is not always sufficient.

The system must also verify whether the user may access the specific object.

Examples:

User has SALE_VIEW
+
Sale belongs to authorized branch
↓
Access allowed

This prevents insecure direct object reference and cross-branch data leakage.

---

37. Security Architecture

Security is layered.

Browser
↓
HTTPS
↓
Authentication
↓
Session Security
↓
Rate Limiting / Abuse Controls
↓
Authorization
↓
Input Validation
↓
Business Rules
↓
Database Constraints
↓
Audit

Security requirements in `PROTECTION.md` take precedence for detailed controls.

The architecture must also protect:

- Secrets
- Sessions
- Files
- APIs
- Database access
- Backups
- Logs
- Background jobs
- Integrations

---

38. File and Upload Security

Files are stored outside normal transactional tables unless there is a specific reason.

ERP
↓
Validate Upload
↓
Store Metadata
↓
Object Storage

Controls should include, as applicable:

- Authentication
- Authorization
- File-type validation
- Size limits
- Safe filenames/object keys
- Malware scanning where required
- Private storage by default
- Signed/controlled access
- Auditability

Files must not become an uncontrolled path to private ERP data.

---

39. Audit Architecture

Critical actions generate audit records.

USER ACTION
↓
AUTHORIZATION
↓
BUSINESS OPERATION
↓
DATABASE CHANGE
↓
AUDIT EVENT

Audit records may include:

- User
- Action
- Entity
- Entity ID
- Timestamp

- Before state where appropriate
- After state where appropriate
- Reason
- Request/correlation ID
- Source transaction

Audit logging must not expose sensitive secrets.

---

40. API Architecture

The frontend communicates with the backend through a documented API boundary.

```
WEB / POS
  ↓ HTTPS
API
  ↓
APPLICATION SERVICES
  ↓
DOMAIN
  ↓
PERSISTENCE
```

The API shall support:

- Authentication
- Authorization
- Validation
- Pagination
- Filtering
- Sorting
- Consistent errors
- Idempotency for appropriate commands
- Request correlation
- Versioning strategy

Exact endpoints belong in `API.md`.

---

41. Command vs Query Separation

The architecture should distinguish between:

Commands

Actions that change state:

- Create sale
- Confirm purchase
- Receive goods
- Transfer stock
- Record payment
- Cancel invoice
- Adjust stock

Queries

Actions that read state:

- Search product
- View stock
- View invoice
- View customer balance
- View report

Commands must enforce business rules and transaction boundaries.

Queries must not mutate business state.

---

42. Idempotency Architecture

Operations that may be retried must be designed to avoid duplicate effects.

Examples:

- Tally sync
- Payment callbacks
- Import operations
- Background jobs
- Certain create commands

A stable business/integration identity should be used where necessary.

Request / Event Identity
↓
Idempotency Check
↓
Process Once
↓

Persist Result

Idempotency must be implemented at the appropriate application/database boundary rather than relying on frontend behavior.

---

43. Background Processing

Background processing is for work that should not block normal ERP operations.

Candidates:

- Tally synchronization
- Notifications
- Large reports
- Exports
- Maintenance
- File processing
- Reconciliation

Application
  ↓
Durable Job/Event Record
  ↓
Worker
  ↓
External Operation

The use of Redis/BullMQ is conditional on actual requirements and the approved technology baseline. A queue is not required merely because the architecture contains asynchronous work.

For critical integration work, the durable job/event state must be stored in PostgreSQL or another explicitly approved durable mechanism.

---

44. Outbox / Integration Event Pattern

When a database transaction must reliably trigger an external action:

BEGIN
  ERP business changes
  Integration/outbox record
COMMIT

↓
Worker reads pending record
↓
External system

This avoids the dangerous sequence:

DB COMMIT
↓
External API call fails
↓
No record that the call was required

The exact outbox implementation belongs in `DATABASE.md` and `TALLY-INTEGRATION.md`.

---

45. TallyPrime Integration Architecture

TallyPrime is isolated behind an integration boundary.

ERP Domain
↓
Integration Record
↓
Tally Sync Worker
↓
Tally Adapter
↓
TallyPrime
↓
Sync Result
↓
Reconciliation

The ERP must not directly depend on Tally availability.

---

46. Tally Sync State

A synchronization lifecycle may be:

PENDING
↓
PROCESSING

↓
SUCCESS

Failure path:

PROCESSING
↓
FAILED
↓
RETRY
↓
PROCESSING

After configured retry limits:

FAILED
↓
MANUAL REVIEW

The exact state names may be finalized in `TALLY-INTEGRATION.md`.

---

47. Tally Idempotency and Duplicate Protection

Duplicate voucher creation is a critical risk.

The integration must maintain stable ERP transaction identity.

ERP Transaction ID
↓
Integration Mapping
↓
Tally Reference / Identity

Retries must determine whether the transaction was already successfully processed.

The ERP must never blindly create a second voucher merely because the previous request timed out.

---

48. Tally Reconciliation

The system must provide visibility into integration state.

```
ERP              TALLY
--------------------------------
INV-1001  ✓       ✓
INV-1002  ✓       ✓
INV-1003  ✓       ✗
INV-1004  ✓       ✓
```

Authorized users should be able to identify:

- Pending
- Failed
- Missing
- Duplicate-risk
- Mismatched
- Successfully synchronized

Exact reconciliation rules belong in `TALLY-INTEGRATION.md`.

---

49. Reporting Architecture

Reports are read-only from the transactional perspective.

```
USER
↓
REPORT REQUEST
↓
REPORT SERVICE
↓
READ MODEL / DATABASE
↓
RESULT
```

Simple reports may execute synchronously.

Large reports should use background processing.

Reporting must not directly modify transactional records.

If reporting load becomes significant, a separate reporting/read model may be introduced later.

---

50. Search Architecture

Initial search should use PostgreSQL capabilities where sufficient.

Search targets may include:

- Product name
- SKU
- Barcode
- Model
- Serial number
- Party name
- Invoice number
- Purchase document number

A dedicated search engine should be introduced only when actual scale or search requirements justify it.

---

51. Caching Architecture

Caching is optional and selective.

Suitable candidates may include:

- Read-heavy configuration
- Product/catalog reads
- Non-sensitive reference data
- Expensive report results

Cache is never authoritative for:

- Stock
- Payment state
- Invoice state
- Financial balances
- Tally synchronization truth

Cache invalidation must not be allowed to create correctness problems.

---

52. Observability Architecture

Production observability should include:

Logs

- Application logs
- Authentication/security logs
- Business operation logs where useful
- Integration logs
- Worker/job logs

Metrics

- Request latency
- Error rate
- Database health
- Worker health
- Job failures
- Tally sync failures
- Inventory operation failures
- Authentication failures

Tracing / Correlation

Requests and asynchronous operations should support correlation identifiers where practical.

Sensitive values must not be logged.

---

53. Error Architecture

Errors should be classified consistently.

Validation Error

Example:

```
SKU already exists.
```

Authentication Error

Example:

```
Invalid credentials.
```

Authorization Error

Example:

```
User is not permitted to perform this operation.
```

Business Rule Error

Example:

```
Insufficient stock.
```

Concurrency Error

Example:

```
Stock changed before this operation completed.
```

Integration Error

Example:

```
Tally is unavailable.
```

System Error

Example:

```
Unexpected database failure.
```

The API must expose a consistent error structure.

Internal stack traces and secrets must not be exposed to clients.

---

54. Database Architecture

PostgreSQL is the primary transactional database.

The database must enforce integrity through:

- Primary keys
- Foreign keys
- Unique constraints
- Not-null constraints where appropriate
- Check constraints where appropriate

- Indexes
- Transaction boundaries
- Appropriate locking/concurrency controls

The database is not merely a storage bucket.

Where safe and practical, invariants should be reinforced at the database layer.

Detailed schema belongs in `DATABASE.md`.

---

55. Data Integrity Rules

The architecture must avoid uncontrolled polymorphic references for critical relationships where strong referential integrity is practical.

Examples requiring deliberate design:

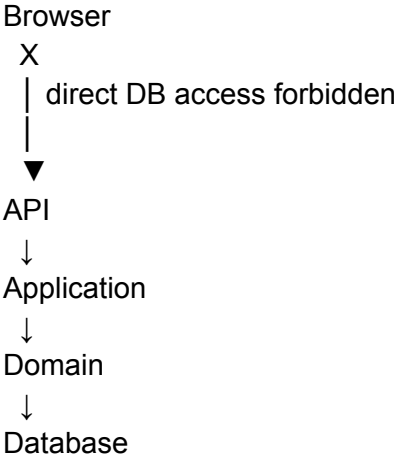
- Payment allocations
- Inventory movement sources
- Serial/batch allocations
- Delivery/installation item links

If polymorphic references are retained, application-level validation and auditability must compensate for the lack of direct foreign-key enforcement.

---

56. API-to-Database Security Boundary

Only authorized server-side application code may access transactional database operations.



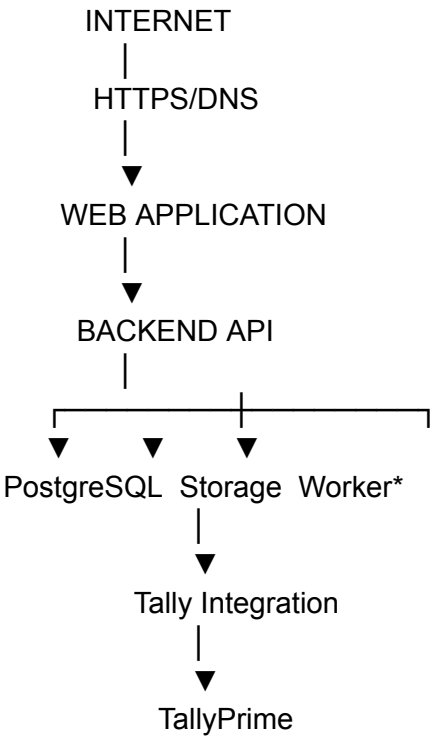
Database credentials must never be exposed to browsers.

---

57. Deployment Architecture

The initial deployment should remain operationally simple.

Conceptually:



* Worker/queue infrastructure is introduced only when justified.

Exact hosting and deployment choices belong in `TECHSTACK_v1.1.md` / deployment documentation.

---

58. Environment Architecture

Separate environments:

```
DEVELOPMENT
    ↓
STAGING
    ↓
```

PRODUCTION

Rules:

- Production credentials are never reused in development.
- Production data must not be casually copied into development.
- Test environments should use synthetic or sanitized data.
- Configuration and secrets must be environment-specific.
- Production deployments should be traceable to a versioned build.

---

59. Backup and Disaster Recovery

Production must have scheduled backups.

The final deployment design must define:

- Backup frequency
- Retention
- Encryption/protection
- Restore procedure
- Restore testing
- RPO
- RTO
- Backup monitoring

Backups are not considered successful merely because a backup job reports success; restoration must be tested.

---

60. Failure Architecture

The system should fail in controlled ways.

Tally Offline

ERP Transaction
↓
COMMIT
↓
Tally Job = PENDING
↓
ERP continues
↓

Retry later

Worker Down

Committed Jobs
↓
Remain durable
↓
Worker Restarts
↓
Jobs Resume

Database Failure

Request
↓
Database Failure
↓
Transaction Not Committed
↓
Safe Error

The system must not report a business transaction as completed when the authoritative transaction did not commit.

External Notification Failure

A sale should not be rolled back simply because an optional notification failed.

---

61. Public Website Boundary

The public website is separate from the ERP application.

PUBLIC WEBSITE

PUBLIC / APPROVED API

PUBLIC-SAFE DATA

The public website must never expose:

- Private ERP records

- Administrative APIs
- Internal financial data
- User credentials
- Internal integration data
- Unauthorized customer information

The public website is secondary to the ERP and should be developed after the core ERP is stable.

---

62. Frontend Architecture

The frontend should be organized by business capability.

APP
├── Dashboard
├── Sales / POS
├── Purchases
├── Inventory
├── Products
├── Parties
├── Payments
├── Returns
├── Fulfilment
├── Delivery
├── Installation
├── Warranty
├── Reports
├── Tally
└── Settings

Frontend responsibilities:

- User experience
- Navigation
- Form handling
- Client-side validation
- Displaying authorization-aware UI
- Optimistic UI only where safe

The frontend must never be the final authority for business correctness.

---

63. POS Architecture

POS should optimize for speed while using the same backend sales engine.

POS UI
↓
Sales API
↓
Sales Domain
↓
Pricing + Inventory + Payments
↓
PostgreSQL

POS must not create a second source of truth.

Performance priorities:

- Fast product search
- Barcode input
- Minimal unnecessary network calls
- Efficient cart updates
- Clear payment flow
- Reliable transaction commit
- Good recovery from network errors

---

64. Transactional UX Principle

The UI should clearly distinguish:

- Draft
- Processing
- Successfully committed
- Failed
- Pending external synchronization

For example, a sale can be:

SALE = COMMITTED
TALLY = PENDING

These are not contradictory states.

The user must not be told that the sale failed merely because Tally is temporarily unavailable.

---

65. Data Import / Export Architecture

Imports and exports must use controlled application workflows.

Imports should:

1. Validate input
2. Authenticate/authorize user
3. Validate business rules
4. Report errors clearly
5. Commit valid changes atomically where appropriate
6. Audit important imports

Bulk imports must not bypass domain rules merely because they are administrative.

Exports must respect authorization and data sensitivity.

---

66. Integration Boundary Rules

Every external integration should have:

- Adapter
- Authentication/configuration
- Timeout handling
- Retry policy where appropriate
- Idempotency strategy
- Error classification
- Audit/logging
- Reconciliation where needed

The domain must not contain vendor-specific HTTP details.

Bad:

Sales Domain → Tally HTTP API

Good:

Sales Domain
↓
Integration Contract
↓
Tally Adapter
↓
TallyPrime

---

67. Scalability Strategy

Scale the simplest component that actually becomes a bottleneck.

Initial strategy:

Modular Monolith
+
PostgreSQL
+
Optional Worker
+
Optional Cache

Future evolution may include:

Reporting Read Model
Notification Worker
Integration Worker
Search Service

Only after measurement and operational need.

---

68. Performance Principles

Performance work must not weaken correctness.

Priorities:

1. Database indexes for real query patterns
2. Efficient product/party search
3. Efficient inventory reads
4. Avoid unnecessary network round trips
5. Paginate large lists
6. Avoid loading huge datasets into memory
7. Use background processing for expensive work
8. Cache only safe read-heavy data
9. Measure before introducing infrastructure

---

69. Architectural Anti-Patterns

The following are prohibited unless explicitly approved:

Direct Frontend → Database

Forbidden.

Business Rules Only in Frontend

Forbidden.

Separate Retail and Wholesale Inventories

Forbidden.

Tally as Operational Source of Truth

Forbidden.

Redis as Inventory Truth

Forbidden.

Blind Tally Retry

Forbidden.

Silent Deletion of Posted Transactions

Forbidden.

Module-to-Module Table Manipulation Without Ownership

Forbidden.

Hard-Coded Brand Logic

Forbidden.

Microservices Before Need

Discouraged.

Uncontrolled Polymorphic References

Discouraged for critical relationships.

Infrastructure for Hypothetical Scale

Discouraged.

---

70. System-of-Truth Matrix

Data	Authoritative Source
Product master	ERP / PostgreSQL
Party master	ERP / PostgreSQL
Price configuration	ERP / PostgreSQL
Stock movement history	ERP / PostgreSQL
Current inventory projection	ERP / PostgreSQL
Sales	ERP / PostgreSQL
Purchases	ERP / PostgreSQL
Payments	ERP / PostgreSQL
Returns	ERP / PostgreSQL
Delivery	ERP / PostgreSQL
Warranty	ERP / PostgreSQL
Audit	ERP / PostgreSQL
Integration state	ERP / PostgreSQL
Accounting ledger	TallyPrime
Binary files	Object Storage
Cache	Non-authoritative
Queue	Non-authoritative transport/work state

---

71. Critical Business Flow — Sale

User / POS
↓

Create Sale
↓
Authenticate + Authorize
↓
Validate Request
↓
Resolve Pricing
↓
Validate Party / Credit Rules
↓
Validate Inventory
↓
BEGIN DATABASE TRANSACTION
↓
Create Sale
↓
Create Sale Items
↓
Deduct Inventory
↓
Update Serial/Batch State
↓
Record Payment / Allocation
↓
Create Audit Record
↓
Create Integration Record
↓
COMMIT
↓
Return Success
↓
Tally Processing Asynchronously

---

72. Critical Business Flow — Purchase Receipt

Purchase / Goods Receipt
↓
Authenticate + Authorize
↓
Validate Product / Quantity
↓
BEGIN TRANSACTION

↓
Create Receipt
↓
Create Inventory Increases
↓
Register Serial / Batch
↓
Create Audit Record
↓
Create Integration Record if applicable
↓
COMMIT

---

73. Critical Business Flow — Return

Original Transaction
↓
Return Request
↓
Authorization
↓
Validate Return Eligibility
↓
Inspection
↓
Disposition
↓
BEGIN TRANSACTION
↓
Inventory / Financial Effects
↓
Payment / Credit Adjustment where applicable
↓
Audit
↓
Integration Record
↓
COMMIT

---

74. Critical Business Flow — Stock Transfer

Transfer Request
↓
Authorization
↓
Validate Source Stock
↓
BEGIN TRANSACTION
↓
Transfer Out
↓
Transfer State
↓
Transfer In / Destination Allocation
↓
Audit
↓
COMMIT

The exact in-transit workflow is a business-rule decision if required.

---

75. Critical Business Flow — Tally Synchronization

ERP Transaction
↓
Committed in PostgreSQL
↓
Integration Record = PENDING
↓
Worker
↓
Tally Adapter
↓
TallyPrime
│
├── Success → SYNCED
│
└── Failure → RETRY
↓
Manual Review

---

76. Security Threat Boundaries

Important trust boundaries include:

Internet
  |
  ▼
Web/API Boundary
  |
  ▼
Authenticated ERP
  |
  ▼
Privileged Business Operations
  |
  ▼
Database

Additional boundaries:

ERP ↔ Tally
ERP ↔ Object Storage
ERP ↔ Notification Providers
ERP ↔ Future Integrations

Each boundary must validate inputs and control credentials.

---

77. Correlation and Traceability

A request or business operation should be traceable across relevant layers.

Conceptually:

Request ID
  ↓
Application Operation
  ↓
Database Transaction
  ↓
Audit Event
  ↓
Background Job
  ↓
External Integration Attempt

This allows operational debugging without relying on guesswork.

---

78. Data Retention and Deletion

Retention policies must distinguish:

- Operational records
- Audit records
- Integration records
- Files
- Logs
- Backups

Important business history must not be deleted simply to clean up the UI.

Any legal or regulatory retention requirements must be incorporated when confirmed.

---

79. Architectural Decision Records

Significant decisions should be recorded as ADRs.

Recommended examples:

ADR-001-modular-monolith.md
ADR-002-postgresql-source-of-truth.md
ADR-003-inventory-ledger.md
ADR-004-tally-asynchronous-integration.md
ADR-005-secure-session-strategy.md
ADR-006-conditional-background-queue.md
ADR-007-public-website-boundary.md

Each ADR should contain:

- Context
- Problem
- Decision
- Alternatives considered
- Reason
- Consequences
- Status

---

80. Key Architectural Risks

Risk 1 — Inventory Inaccuracy

Mitigation: Central inventory engine, movement ledger, database transactions, concurrency controls.

Risk 2 — Duplicate Tally Transactions

Mitigation: Stable integration identity, idempotency, retry controls, reconciliation.

Risk 3 — Permission Bypass

Mitigation: Server-side authorization and object-level access control.

Risk 4 — Overengineering

Mitigation: Modular monolith and conditional infrastructure.

Risk 5 — Scope Creep

Mitigation: Phase-based development and explicit change control.

Risk 6 — Database Design Weakness

Mitigation: Strong relational constraints, ERD review, transaction-boundary review.

Risk 7 — POS Slowness

Mitigation: Search/index optimization and minimal synchronous dependencies.

Risk 8 — External Integration Failure

Mitigation: Durable integration state, retries, manual reconciliation.

Risk 9 — Data Loss

Mitigation: Backups, restore testing, transaction integrity, disaster recovery planning.

Risk 10 — Cross-Branch Data Leakage

Mitigation: Server-side organization/branch authorization and object-level access checks.

---

81. Architectural Constraints

The initial system shall:

1. Use a modular monolith.
2. Use PostgreSQL as the transactional source of truth.
3. Keep Tally outside the ERP transaction boundary.
4. Use one inventory engine for retail, wholesale, and dealer operations.
5. Enforce business rules server-side.
6. Enforce critical integrity in the database where practical.
7. Keep important posted history.
8. Avoid hard-coded business identities.
9. Keep external integrations behind adapters/boundaries.
10. Introduce queues/cache/search services only when justified.
11. Preserve auditability for critical operations.
12. Protect branch/location data by authorization scope.

---

82. Architecture Success Criteria

The architecture is acceptable when:

- Core business transactions are atomic.
- Inventory is traceable.
- Concurrent stock operations are safe.
- Retail and wholesale share the same inventory engine.
- Payment allocation is consistent.
- Posted transaction history is preserved.
- Authorization is server-enforced.
- Branch/location access is controlled.
- Tally failure does not block valid ERP operations.
- Tally synchronization is retryable and reconcilable.
- External integrations are isolated.
- Production backups can be restored.
- The public website cannot access private ERP data.
- Modules remain understandable and independently testable.
- Infrastructure remains proportionate to actual scale.
- The architecture can evolve without an immediate rewrite.

---

83. Architecture Roadmap

Phase 1 — Core ERP

Modular Monolith
+
PostgreSQL
+
Core Modules
+
Inventory Engine
+
Sales/POS
+
Purchasing
+
Payments
+
Audit

Phase 2 — Integration

Tally Integration
+
Background Processing
+
Reconciliation
+
Notifications

Phase 3 — Operational Expansion

Delivery
+
Installation
+
Warranty
+
Advanced Reporting

Phase 4 — Website

Public Website
+
Controlled Public API
+

Catalog / Content

Future Only If Justified

Read Model
Search Service
Dedicated Integration Service
Notification Service
Other Extracted Services

---

84. Relationship With Other Project Documents

BRD.md
↓
Business Requirements
↓
SRS.md
↓
Software Requirements
↓
ARCHITECTURE.md
↓
System Structure
↓
PROTECTION.md
↓
Security Requirements
↓
TECHSTACK_v1.1.md
↓
Technology Choices
↓
BUSINESS-RULES.md
↓
Authoritative Business Policies
↓
DATABASE.md
↓
Data Model
↓
ERD
↓
API.md

↓
API Contracts
↓
TALLY-INTEGRATION.md
↓
Accounting Integration
↓
DESIGN.md
↓
UI/UX
↓
PHASES.md
↓
Implementation Roadmap

No single document should become a substitute for the others.

---

85. Open Architectural Decisions

The following require confirmation or implementation-level decisions:

1. Exact authentication/session implementation.
2. Exact queue/background worker implementation.
3. Whether Redis is required in the first deployment.
4. Exact TallyPrime communication mechanism.
5. Tally voucher/master synchronization scope.
6. Inventory valuation method.
7. Negative-stock policy.
8. Credit-limit enforcement behavior.
9. GST/tax calculation rules.
10. Invoice numbering rules.
11. Warranty start-date policy.
12. Exact approval thresholds.
13. Branch/location permission model details.
14. Reporting read-model strategy.
15. File-storage provider.
16. Backup RPO/RTO targets.
17. Production hosting topology.
18. Exact observability/error-tracking provider.

Unknown business policies must not be silently invented.

Use:

```
TBD — Business Confirmation Required
```

until approved.

---

86. Implementation Order

The architecture implies the following implementation dependency order:

1. Project foundation
2. Authentication / authorization
3. Organization / branches / warehouses
4. Product / party masters
5. Pricing
6. Inventory engine
7. Procurement
8. Sales / POS
9. Payments
10. Returns
11. Fulfilment / delivery
12. Installation / warranty
13. Audit / reporting
14. Tally integration
15. Notifications
16. Public website

Inventory should be implemented before sales/purchase flows are considered complete because those flows depend on inventory correctness.

---

87. Final Architectural Rules

These rules are considered non-negotiable unless an ADR explicitly changes them:

1. **PostgreSQL is authoritative for ERP transactions.**
2. **Inventory changes go through the inventory engine.**
3. **The frontend is never the authority for business rules.**
4. **Authorization is enforced server-side.**
5. **Critical transactions are atomic.**
6. **Concurrency must not permit duplicate stock consumption.**
7. **Tally availability must not block valid ERP operations.**
8. **Tally synchronization must be retryable and idempotent.**
9. **Posted history must not be silently deleted.**
10. **Retail and wholesale use the same underlying sales/inventory domains.**
11. **External systems are isolated behind integration boundaries.**

12. **Security controls apply to APIs, files, database access, jobs, and integrations.**
13. **Infrastructure is added because of a demonstrated need, not fashion.**
14. **Business policies that are not confirmed remain explicitly TBD.**
15. **Architecture changes that affect system invariants must be recorded as ADRs.**

---

88. Document Status

Version: 2.0
Status: Draft Architecture Baseline — Ready for Cross-Document Review

Supersedes: `ARCHITECTURE.md` v1.0

Next Review Against:

- `SRS.md`
- `PROTECTION.md`
- `TECHSTACK_v1.1.md`
- `BUSINESS-RULES.md`
- `DATABASE.md`
- `API.md`